

STA623 - Bayesian Data Analysis

Practical 3: Monte Carlo, and writing your own MCMC samplers (with solutions)

Marc Henrion

2026-09-30

Goals. Quantify Monte Carlo error; use simulation for posterior predictive checks; implement a Metropolis sampler for a two-parameter model and a Gibbs sampler for the normal model; assess your own samplers with multiple chains. Accompanies Lecture 3.

Exercise 1: Monte Carlo error

The posterior for the malaria-episode rate in Lecture 2 was $\lambda \mid \mathbf{y} \sim \text{Gamma}(45, 20)$.

- (a) For $S = 10, 100, 1000, 10^4, 10^5$, estimate $E[\lambda \mid \mathbf{y}]$ and $P(\lambda < 2.1 \mid \mathbf{y})$ by Monte Carlo, together with their Monte Carlo standard errors (MCSE). Compare with the exact values.
- (b) How many draws are needed for the MCSE of $P(\lambda < 2.1 \mid \mathbf{y})$ to be below 0.005 (so that the second decimal place is reliable)? Derive it from the formula, then check by simulation.
- (c) Repeat the $S = 1000$ estimate of the mean 500 times. Compare the standard deviation of the 500 estimates with the MCSE formula.

Solution

(a)

```
exact <- c(mean = 45 / 20, prob = pgamma(2.1, 45, 20))
map_dfr(c(10, 100, 1000, 1e4, 1e5), function(S) {
  d <- rgamma(S, 45, 20); p <- mean(d < 2.1)
  tibble(S = S, mean = mean(d), mcse_mean = sd(d) / sqrt(S),
         prob = p, mcse_prob = sqrt(p * (1 - p) / S))
}) |> kable(digits = 4)
```

S	mean	mcse_mean	prob	mcse_prob
1e+01	2.1552	0.0765	0.5000	0.1581
1e+02	2.2198	0.0293	0.4000	0.0490
1e+03	2.2456	0.0106	0.3380	0.0150

S	mean	mcse_mean	prob	mcse_prob
1e+04	2.2514	0.0034	0.3412	0.0047
1e+05	2.2508	0.0011	0.3411	0.0015

```
exact
```

```
      mean      prob
2.2500000 0.3417987
```

(b) $\text{MCSE} = \sqrt{p(1-p)/S} \leq 0.005 \iff S \geq p(1-p)/0.005^2.$

```
p <- exact["prob"]
ceiling(p * (1 - p) / 0.005^2)
```

```
prob
8999
```

About 9,000 draws; for any probability, $S = 10,000$ suffices because $p(1-p) \leq 0.25$.

(c)

```
est <- replicate(500, mean(rgamma(1000, 45, 20)))
c(sd_of_estimates = sd(est), mcse_formula = sqrt(45 / 20^2) / sqrt(1000))
```

```
sd_of_estimates    mcse_formula
      0.01045858      0.01060660
```

The empirical spread of repeated estimates matches the MCSE formula.

Exercise 2: posterior predictive simulation and a first model check

Continue with the GSS data of Practical 2 (women without a degree, posterior $\theta_1 \sim \text{Gamma}(219, 112)$). The observed distribution of the number of children in this group was:

```
obs_counts <- tibble(children = 0:6, mothers = c(20, 19, 38, 20, 10, 2, 2))
kable(obs_counts)
```

children	mothers
0	20
1	19
2	38
3	20
4	10
5	2
6	2

```
y_obs <- rep(obs_counts$children, obs_counts$mothers)
```

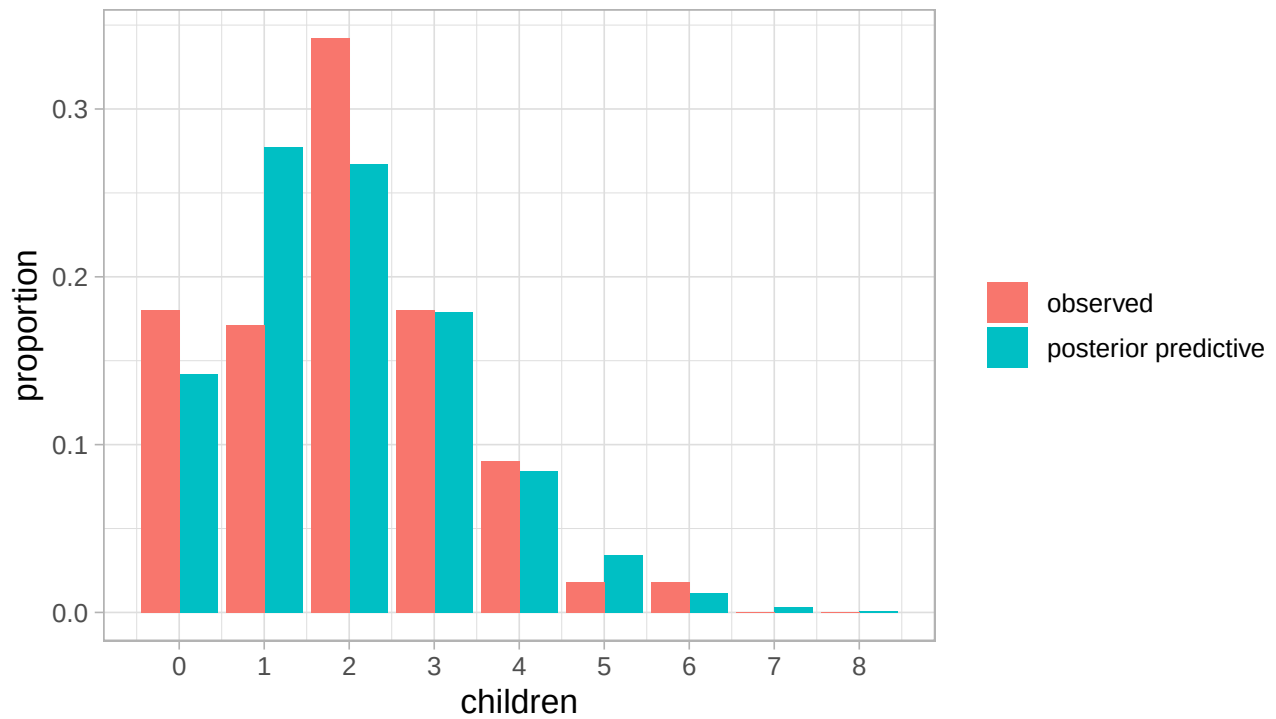
- (a) Simulate the posterior predictive distribution for the number of children of one new woman and plot it next to the observed relative frequencies.
- (b) Let $t(\mathbf{y})$ be the number of women with exactly 2 children divided by the number with exactly 1 child; here $t(\mathbf{y}) = 38/19 = 2$. Simulate 10,000 **replicated datasets** of 111 women, each using a *new* posterior draw of θ_1 , and compute t for each. Estimate $P(t(\mathbf{y}^{\text{rep}}) \geq 2 \mid \mathbf{y})$.
- (c) What does this say about the Poisson model?

Solution

(a)

```
S <- 1e5
th <- rgamma(S, 219, 112)
y_new <- rpois(S, th)

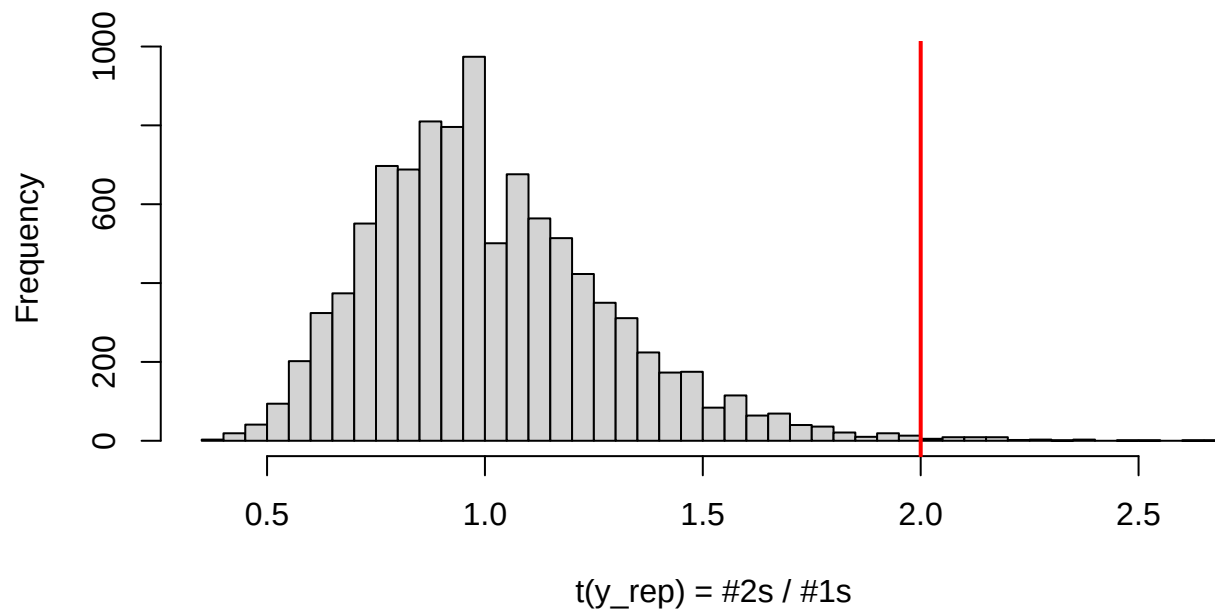
tibble(children = 0:8) |>
  mutate(`posterior predictive` = map_dbl(children, ~ mean(y_new == .x)),
         observed = map_dbl(children, ~ mean(y_obs == .x))) |>
  pivot_longer(-children) |>
  ggplot(aes(children, value, fill = name)) + geom_col(position = "dodge") +
  scale_x_continuous(breaks = 0:8) + labs(y = "proportion", fill = NULL)
```



The model predicts too few women with 2 children and too many with 1.

(b)

```
t_rep <- replicate(10000, {
  y_rep <- rpois(111, rgamma(1, 219, 112))      # new theta for each replicated
  ↪ dataset
  sum(y_rep == 2) / sum(y_rep == 1)
})
hist(t_rep, breaks = 50, main = "", xlab = "t(y_rep) = #2s / #1s"); abline(v = 2,
  ↪ col = "red", lwd = 2)
```



```
mean(t_rep >= 2)
```

```
[1] 0.0056
```

- (c) Replicated datasets almost never show as many 2-child families relative to 1-child families as the real data. The Poisson model captures the mean but not the shape of the distribution (family size is influenced by preferences for two children). This is a posterior predictive check; Lecture 5 develops the idea.

Exercise 3: a Metropolis sampler for Poisson regression

Malaria episodes for 60 children, with an indicator of whether the household has an insecticide-treated net:

```
set.seed(31)
itn <- rep(0:1, each = 30)
episodes <- rpois(60, exp(0.8 - 0.6 * itn))
tapply(episodes, itn, mean)
```

```
      0      1
2.000000 1.233333
```

Model: $Y_i \sim \text{Pois}(\mu_i)$, $\log \mu_i = \alpha_0 + \alpha_1 \text{itn}_i$, priors $\alpha_0 \sim \mathcal{N}(0, 1.5^2)$, $\alpha_1 \sim \mathcal{N}(0, 1)$.

- Write a function `log_post(alpha)` returning the log of likelihood $\times$ prior for $\alpha = (\alpha_0, \alpha_1)$.
- Write a random-walk Metropolis sampler that proposes $\alpha^* \sim \mathcal{N}_2(\alpha^{(s-1)}, \delta^2 \mathbf{I})$. Run 4 chains of 5,000 iterations from dispersed starting values with $\delta = 0.15$. Report the acceptance rate.
- Plot trace plots of all chains. Discard a burn-in of 1,000 iterations, then compute $\hat{R}$ and bulk ESS with the `posterior` package.
- Report the posterior median and 95% credible interval of the rate ratio e^{α_1} and $P(e^{\alpha_1} < 1 \mid \mathbf{y})$. Compare with `glm(episodes ~ itn, family = poisson)`.
- Try $\delta = 0.01$ and $\delta = 1$. What happens to the acceptance rate and ESS?

Solution

(a)

```
log_post <- function(alpha) {
  mu <- exp(alpha[1] + alpha[2] * itn)
  sum(dpois(episodes, mu, log = TRUE)) +
    dnorm(alpha[1], 0, 1.5, log = TRUE) + dnorm(alpha[2], 0, 1, log = TRUE)
}
```

(b)

```
metropolis2 <- function(n_iter, init, step) {
  out <- matrix(NA, n_iter, 2, dimnames = list(NULL, c("alpha0", "alpha1")))
  out[1, ] <- init; lp <- log_post(init); acc <- 0
  for (s in 2:n_iter) {
    prop <- out[s - 1, ] + rnorm(2, 0, step)
    lp_prop <- log_post(prop)
    if (log(runif(1)) < lp_prop - lp) { out[s, ] <- prop; lp <- lp_prop; acc <-
      acc + 1 }
  }
```

```

    else out[s, ] <- out[s - 1, ]
  }
  attr(out, "acceptance") <- acc / (n_iter - 1)
  out
}

inits <- list(c(-2, 2), c(2, -2), c(-2, -2), c(2, 2))
set.seed(32)
chains <- map(inits, ~ metropolis2(5000, .x, step = 0.15))
map_dbl(chains, ~ attr(.x, "acceptance"))

```

```
[1] 0.4862973 0.5089018 0.5015003 0.4822965
```

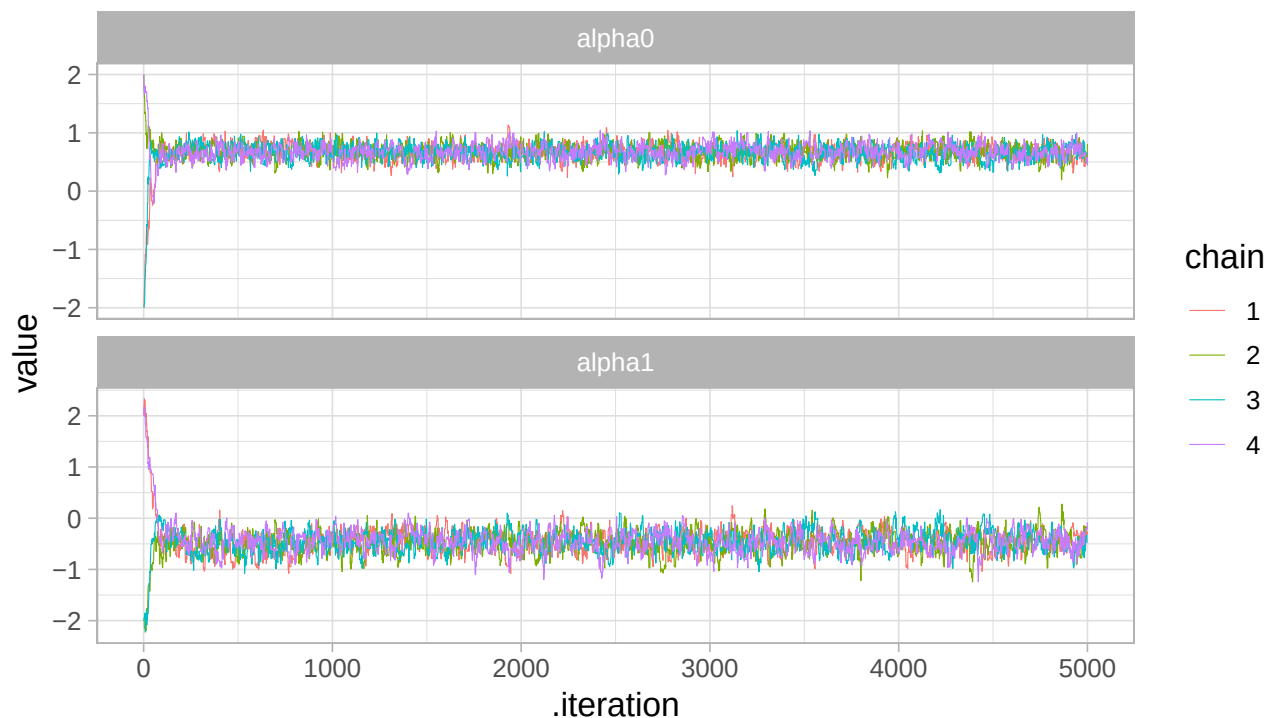
(c)

```

draws <- map_dfr(seq_along(chains), ~ as_tibble(chains[[.x]]) |>
  mutate(.chain = .x, .iteration = row_number()))

draws |> pivot_longer(c(alpha0, alpha1)) |>
  ggplot(aes(.iteration, value, colour = factor(.chain))) + geom_line(linewidth =
    ↪ 0.2) +
  facet_wrap(~name, ncol = 1, scales = "free_y") + labs(colour = "chain")

```



```

post_draws <- draws |> filter(.iteration > 1000) |> as_draws_df()
summarise_draws(post_draws, "median", ~quantile(.x, c(0.025, 0.975)), "rhat",
  ↪ "ess_bulk")

```

```
# A tibble: 2 x 6
  variable median `2.5%` `97.5%` rhat ess_bulk
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl>
1 alpha0    0.671  0.414  0.910  1.00   893.
2 alpha1   -0.452 -0.845 -0.0619 1.00   817.
```

All chains quickly leave their starting points and overlap; $\hat{R} \approx 1.00$.

(d)

```
rr <- exp(post_draws$alpha1)
c(quantile(rr, c(0.5, 0.025, 0.975)), `P(RR < 1)` = mean(rr < 1))
```

```
50%      2.5%      97.5% P(RR < 1)
0.6365630 0.4297237 0.9399734 0.9881875
```

```
exp(cbind(estimate = coef(glm(episodes ~ itn, family = poisson)),
          confint.default(glm(episodes ~ itn, family = poisson))))
```

```
          estimate      2.5 %      97.5 %
(Intercept) 2.0000000 1.5528888 2.5758444
itn          0.6166667 0.4093771 0.9289181
```

With weakly informative priors and a reasonable amount of data, the Bayesian and maximum likelihood results are very similar; the Bayesian analysis additionally gives $P(RR < 1 \mid \mathbf{y})$ directly.

(e)

```
set.seed(33)
map_dfr(c(0.01, 0.15, 1), function(d) {
  ch <- map(inits, ~ metropolis2(5000, .x, step = d))
  dd <- map_dfr(seq_along(ch), ~ as_tibble(ch[[.x]]) |> mutate(.chain = .x,
    ↪ .iteration = row_number())) |>
    filter(.iteration > 1000) |> as_draws_df()
  tibble(step = d, acceptance = mean(map_dbl(ch, ~ attr(.x, "acceptance"))),
    rhat_alpha1 = rhat(extract_variable_matrix(dd, "alpha1")),
    ess_alpha1 = ess_bulk(extract_variable_matrix(dd, "alpha1")))
}) |> kable(digits = 3)
```

step	acceptance	rhat_alpha1	ess_alpha1
0.01	0.919	1.835	5.754
0.15	0.502	1.008	790.956
1.00	0.039	1.011	336.237

A tiny step is accepted almost always but moves so slowly that ESS is tiny and chains may not even have converged ($\hat{R}$ well above 1.01). A huge step is almost always rejected, again giving low ESS. The intermediate step size is by far the most efficient.

Exercise 4: a Gibbs sampler for the normal model

Haemoglobin (g/dL) for 25 pregnant women:

```
hb <- c(10.2, 11.8, 9.4, 12.1, 10.9, 11.3, 8.7, 10.5, 12.6, 11.0, 9.9, 10.8, 11.5,
        10.1, 9.2, 12.0, 10.7, 11.2, 9.8, 10.4, 13.1, 10.0, 11.6, 9.5, 10.9)
```

Model: $Y_i \sim \mathcal{N}(\mu, \sigma^2)$ with independent priors $\mu \sim \mathcal{N}(12, 2^2)$, $1/\sigma^2 \sim \text{Gamma}(\nu_0/2, \nu_0\sigma_0^2/2)$ with $\nu_0 = 1$, $\sigma_0^2 = 2$.

- Write down the two full conditional distributions (see Lecture 2) and implement a Gibbs sampler.
- Run 4 chains of 3,000 iterations from dispersed starting values; check convergence with trace plots and $\widehat{R}$.
- Anaemia in pregnancy is defined as Hb < 11 g/dL. Estimate $P(\mu < 11 \mid \mathbf{y})$, and the posterior predictive probability that a *new* woman from this population is anaemic. Why do these differ?
- Estimate the posterior median and 95% interval of the coefficient of variation σ/μ .

Solution

- $\mu \mid \sigma^2, \mathbf{y} \sim \mathcal{N}(\mu_n, \tau_n^2)$ with $1/\tau_n^2 = 1/\tau_0^2 + n/\sigma^2$, $\mu_n = \tau_n^2(\mu_0/\tau_0^2 + n\bar{y}/\sigma^2)$; and $1/\sigma^2 \mid \mu, \mathbf{y} \sim \text{Gamma}\left(\frac{\nu_0+n}{2}, \frac{\nu_0\sigma_0^2 + \sum(y_i - \mu)^2}{2}\right)$.

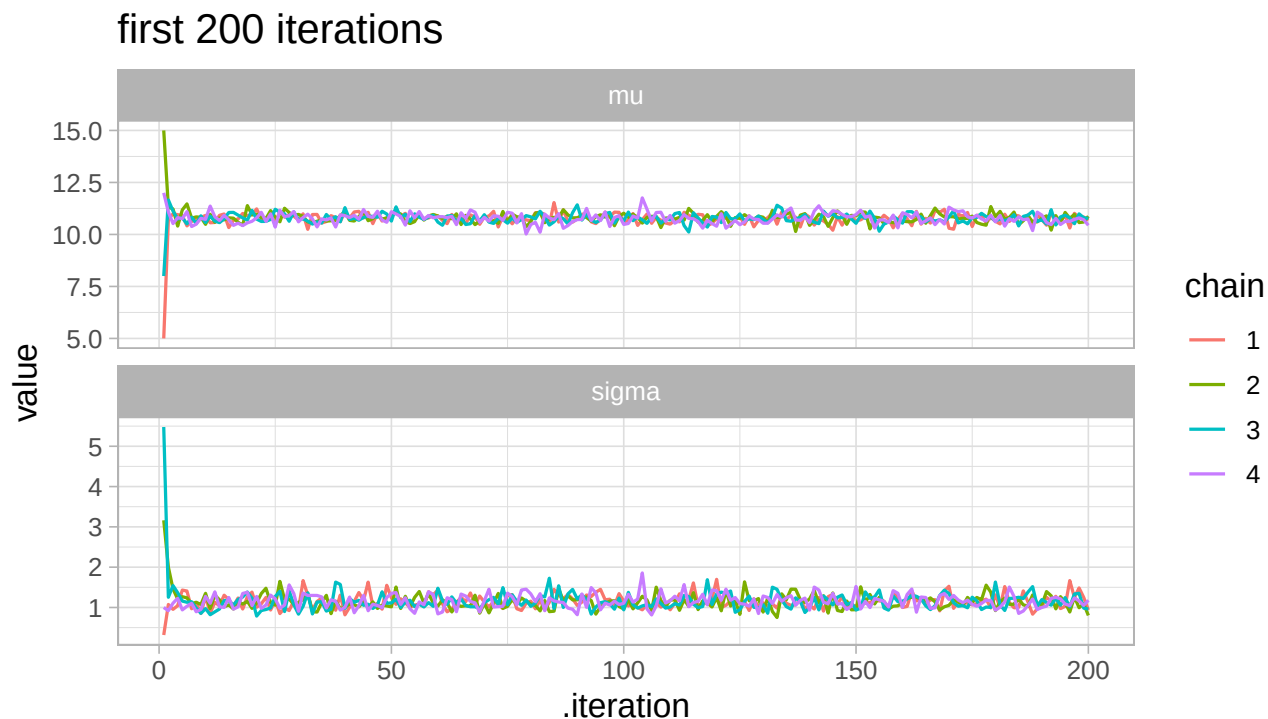
```
gibbs <- function(y, S, mu_init, s2_init, mu0 = 12, t02 = 4, nu0 = 1, s02 = 2) {
  n <- length(y); yb <- mean(y)
  mu <- s2 <- numeric(S); mu[1] <- mu_init; s2[1] <- s2_init
  for (s in 2:S) {
    tn2 <- 1 / (1 / t02 + n / s2[s - 1])
    mu[s] <- rnorm(1, tn2 * (mu0 / t02 + n * yb / s2[s - 1]), sqrt(tn2))
    s2[s] <- 1 / rgamma(1, (nu0 + n) / 2, rate = (nu0 * s02 + sum((y - mu[s])^2))
    ↪ / 2)
  }
  tibble(mu = mu, sigma = sqrt(s2))
}
```

-

```
set.seed(41)
starts <- list(c(5, 0.1), c(15, 10), c(8, 30), c(12, 1))
gdraws <- map_dfr(seq_along(starts), ~ gibbs(hb, 3000, starts[[.x]][1],
    ↪ starts[[.x]][2]) |>
  mutate(.chain = .x, .iteration = row_number()))

gdraws |> filter(.iteration <= 200) |> pivot_longer(c(mu, sigma)) |>
```

```
ggplot(aes(.iteration, value, colour = factor(.chain))) + geom_line() +
  facet_wrap(~name, ncol = 1, scales = "free_y") + labs(colour = "chain", title =
    ↪ "first 200 iterations")
```



```
gpost <- gdraws |> filter(.iteration > 500) |> as_draws_df()
summarise_draws(gpost, "mean", ~quantile(.x, c(0.025, 0.975)), "rhat", "ess_bulk")
```

```
# A tibble: 2 x 6
  variable mean `2.5%` `97.5%` rhat ess_bulk
  <chr>    <dbl> <dbl>    <dbl> <dbl>    <dbl>
1 mu      10.8  10.3     11.2  1.00     9780.
2 sigma    1.14  0.866    1.53  1.000    9072.
```

The Gibbs sampler forgets even very poor starting values within a few iterations; after discarding 500 iterations $\hat{R} \approx 1.00$.

(c)

```
y_new <- rnorm(nrow(gpost), gpost$mu, gpost$sigma)
c(`P(mu < 11 | y)` = mean(gpost$mu < 11), `P(new woman anaemic | y)` = mean(y_new
  ↪ < 11))
```

$P(\mu < 11 \mid y)$	$P(\text{new woman anaemic} \mid y)$
0.8322	0.5787

$P(\mu < 11)$ is about the *population mean*, which is quite well determined. The predictive probability is about an *individual*, whose Hb varies around the mean with SD $\sigma \approx 1$: it is the estimated prevalence of anaemia, a very different (and here more relevant) quantity.

(d)

```
quantile(gpost$sigma / gpost$mu, c(0.5, 0.025, 0.975))
```

50%	2.5%	97.5%
0.10362581	0.08020556	0.14222369